hexagone
École Supérieure d'Informatique

Git et GitHub

1GEST – Gestion de projet et outils collaboratifs

Chris CHEVALIER

hexagone
École Supérieure d'Informatique

Sommaire

#1 Pourquoi versionner ?

#2 Les notions de Git

#3 GitHub Desktop pas à pas

#4 Travailler en équipe sur GitHub

#5 Souveraineté numérique

#1 Pourquoi versionner ?

Git et GitHub

Vous connaissez déjà ce dossier

Dossier « Projet »

```
rapport.docx
rapport-v2.docx
rapport-v2-relu-par-Manon.docx
rapport-final.docx
rapport-final-v3.docx
rapport-final-v3-VRAIMENT-final.docx
```

- Quelle est la **bonne** version ?
- Qu'est-ce qui a changé entre deux versions ?
- Qui a modifié quoi ? Quand ? Pourquoi ?
- Comment travailler à trois sur le même fichier **sans écraser** le travail des autres ?

Le **versionnement** répond à toutes ces questions. Il enregistre chaque étape d'un projet avec son auteur, sa date et la raison du changement.

Git et GitHub : deux choses différentes

Git · le logiciel

- Installé **sur votre ordinateur**
- Enregistre l'historique de votre projet
- **Libre et gratuit**, créé en 2005 par Linus Torvalds
- **Distribué** : chaque membre possède une copie complète du projet et de son historique

GitHub · l'hébergeur

- Un **site web** qui stocke une copie du projet en ligne
- Partage le projet avec l'équipe
- Ajoute des outils de travail en équipe : tickets, relecture, tableaux
- Appartient à **Microsoft** depuis 2018

Logiciel libre : logiciel dont le code est public et que chacun peut utiliser, étudier, modifier et redistribuer. Linus Torvalds est aussi le créateur du noyau Linux. D'autres hébergeurs existent : GitLab, Codeberg…

#2 Les notions de Git

Git et GitHub

Dépôt local et dépôt distant

Dépôt local

Le dossier du projet **sur votre ordinateur**, avec tout son historique. Vous y travaillez, même sans Internet.

Synchronisation

Push ⟶ envoyer vos étapes en ligne

Pull ⟵ récupérer celles de l'équipe

Dépôt distant

La copie du projet **sur GitHub**. C'est la référence commune que toute l'équipe partage.

Dépôt (*repository*, souvent abrégé en *repo*) : dossier de projet suivi par Git. Le dépôt distant s'appelle par défaut `origin`.

Le commit : une photo de votre projet

- Un **commit** est une photo de votre projet à un instant donné, accompagnée d'un message qui résume ce qui a changé
- Chaque commit garde son **auteur**, sa **date** et un **identifiant unique**
- On peut revenir à n'importe quelle photo de l'historique

Message	Avis
« modifs »	✗
« update README.md »	✗
« Ajout des user stories du sprint 1 »	✓
« Correction de l'estimation de la recherche »	✓

> Un bon message dit **la nature du changement**. En lisant seulement l'historique, on doit comprendre ce qui s'est passé sans ouvrir le commit.

Le cycle de travail

1 · Pull

- Récupérer le travail de l'équipe
- **Toujours avant de commencer**

2 · Modifier

- Écrire, corriger, ajouter des fichiers
- Enregistrer dans l'éditeur

3 · Commit

- Choisir les fichiers
- Écrire un message clair
- Valider l'étape

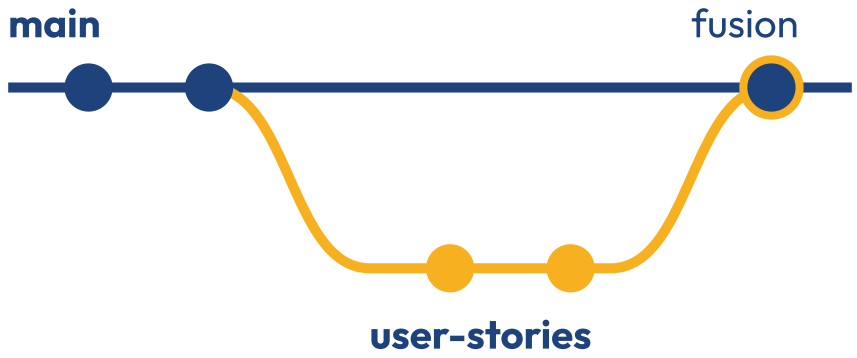
4 · Push

- Envoyer vos commits sur GitHub
- L'équipe peut les récupérer

> Un commit reste **sur votre ordinateur** tant que vous ne l'avez pas poussé. Pour l'équipe, un travail non poussé n'existe pas.

Les branches

- Une **branche** est une ligne de travail indépendante. Vous y avancez sans perturber les autres
- La branche principale s'appelle `main`. Elle contient la version **validée** du projet
- Une branche par tâche, avec un nom clair : `user-stories`, `fix-readme-links`



> Quand la tâche est terminée, on **fusionne** la branche dans `main`. On parle de **merge**.

La pull request : faire relire avant de fusionner

- Une **pull request** (PR) est une demande de fusion d'une branche dans `main`, faite sur GitHub
- Un coéquipier **relit** les changements, commente puis approuve
- La fusion se fait sur GitHub avec le bouton **Merge pull request**

Exemple

Manon ouvre la PR « Ajout des user stories du sprint 1 ».

Cosmo la relit et signale une story sans critère d'acceptation. Manon corrige sur sa branche : la PR se met à jour toute seule.

Cosmo approuve, Chris fusionne.

Sur GitLab, on parle de *merge request* : c'est la même chose. Documentation GitHub sur les pull requests

Les conflits

- Un **conflit** apparaît quand deux personnes modifient **la même ligne** du même fichier
- Git ne peut pas deviner la bonne version. Il vous demande de choisir
- Ce n'est **pas une erreur** : c'est une décision à prendre, à deux de préférence

README.md (en conflit)

```
<<<<<<< HEAD
# Appli de covoiturage étudiant
=======
# CoVoit'Campus
>>>>>>> titre-projet
```

> **Résoudre** : garder une version, l'autre ou un mélange des deux, **supprimer les lignes de marqueurs** (`<<<` , `===` , `>>>`), enregistrer puis faire un commit.

Markdown : la syntaxe en une diapo

Markdown : du texte simple et quelques symboles. Fichier en `.md`, affiché mis en forme par GitHub.

J'écris	J'obtiens
`# Titre`	le titre du document (un seul)
`## Section` · `### Sous-section`	des titres de section
`**important** · *nuance*`	**important** · *nuance*
`- élément`	une liste à puces
`1. étape`	une liste numérotée
`- [ ] à faire` · `- [x] fait`	des cases à cocher

J'écris	J'obtiens
`[Trello](https://trello.com)`	un lien cliquable
`![Logo](docs/logo.png)`	une image
`` `docs/need.md` ``	un nom de fichier ou une commande
`> Citation`	un paragraphe en retrait
`| A | B | puis |---|---|`	un tableau

Une ligne vide sépare deux paragraphes. Toute la syntaxe : écriture et mise en forme sur GitHub. L'onglet **Preview** de l'éditeur de GitHub montre le résultat avant d'enregistrer.

Le README : la page d'accueil du dépôt

GitHub l'affiche sous la liste des fichiers : c'est la **première chose** qu'on lit en ouvrant votre dépôt.

En une minute, il répond à quatre questions :

- **Quoi ?** Le projet en deux phrases
- **Pour qui et pourquoi ?** Le client et son besoin
- **Où en est-on ?** L'état et les liens utiles
- **Qui ?** L'équipe et le rôle de chacun

Les sections de votre README

1. Titre et présentation du projet
2. Le client et son besoin
3. L'équipe et les rôles (tableau)
4. Les liens : Trello, projet GitHub
5. Les documents de `docs/` (liens)
6. L'avancement (cases à cocher)
7. La déclaration d'usage de l'IA

> Un README se lit en diagonale : titres clairs, listes courtes, liens.

Un README complet

README.md

```
# Grand Prix Hexagone

Application web du tournoi Mario Kart de la rentrée :
inscriptions, grille des courses et classement en direct.

## Le client et son besoin

Le **bureau des étudiants** veut en finir avec le papier.
Détails : [docs/need.md](docs/need.md)

## L'équipe

| Membre | Rôle           |
|--------|----------------|
| Manon  | Product Owner  |
| Cosmo  | Scrum Master   |
| Chris  | Developer      |

## Liens

- [Tableau Trello](https://trello.com/b/…)
- [Projet GitHub](https://github.com/…/projects/1)

## Avancement

- [x] Besoin reformulé et validé
- [x] Backlog estimé
- [ ] Sprint 1 terminé

## Usage de l'IA

Le Chat (Mistral AI) a relu `docs/need.md`.
```

> Un seul `#`, une section `##` par question, des **liens** plutôt que des copier-coller.

#3 GitHub Desktop pas à pas

Git et GitHub

Installer et se connecter

1. Créez un compte sur github.com et activez la **double authentification**
2. Téléchargez **GitHub Desktop** (Windows ou macOS) et installez-le
3. Au premier lancement, cliquez sur **Sign in to GitHub.com**. Le navigateur s'ouvre pour autoriser l'application
4. Vérifiez votre nom et votre adresse e-mail. Ils signeront chacun de vos commits

> **GitHub Desktop** est une application qui utilise Git sans taper de commande. Chaque bouton lance une commande Git à votre place.

Double authentification (2FA) : en plus du mot de passe, un code temporaire donné par une application du téléphone. Un mot de passe volé ne suffit plus pour entrer dans votre compte.

Récupérer le dépôt de l'équipe : cloner

1. Un membre crée le dépôt sur github.com (**New repository**) en cochant **Add a README file**
2. Il invite les autres membres (diapo suivante)
3. Chacun ouvre GitHub Desktop : **File** → **Clone repository**
4. Choisissez le dépôt dans la liste, puis le dossier où le ranger sur votre ordinateur
5. Cliquez sur **Clone**. Le projet et tout son historique arrivent sur votre machine

Cloner : télécharger une copie complète d'un dépôt distant, historique compris. On ne le fait qu'une fois par machine.

Inviter les membres de l'équipe

1. Sur github.com, ouvrez le dépôt
2. **Settings** → **Collaborators**
3. **Add people**, puis saisissez le nom d'utilisateur de chaque membre
4. Chaque membre accepte l'invitation reçue par e-mail

Documentation GitHub : inviter des collaborateurs

> Invitez aussi **Chris-CHEVALIER**. Je suis votre projet depuis le dépôt et c'est lui que je note.

Faire un commit puis le pousser

1. Modifiez un fichier puis enregistrez-le
2. Dans GitHub Desktop, l'onglet **Changes** liste les fichiers modifiés. Les lignes ajoutées sont en vert, les lignes supprimées en rouge
3. Cochez les fichiers à inclure
4. Remplissez **Summary** (le message) et, si besoin, **Description**
5. Cliquez sur **Commit to main** (le nom de la branche courante)
6. Cliquez sur **Push origin**

> **Avant de commencer** : cliquez sur **Fetch origin**. S'il y a du nouveau, le bouton devient **Pull origin** : cliquez dessus.

L'onglet **History** affiche tous les commits de l'équipe, avec leur auteur, leur date et leurs changements.

Créer une branche puis ouvrir une pull request

1. **Current Branch** → **New Branch**, un nom clair, puis **Create Branch**
2. Faites vos commits sur cette branche
3. **Publish branch** l'envoie sur GitHub
4. **Preview Pull Request**, vérifiez **base: main** puis **Create Pull Request**
5. Dans le navigateur, donnez un titre clair puis **Create pull request**

Sur github.com

- Un coéquipier ouvre l'onglet **Files changed** puis **Review changes**
- Il commente ou approuve (**Approve**)
- On fusionne avec **Merge pull request**
- Chacun revient sur `main` et fait un **Pull**

Documentation : créer une pull request

Résoudre un conflit dans GitHub Desktop

1. GitHub Desktop signale les fichiers en conflit (par exemple en mettant à jour votre branche avec `main`)
2. Ouvrez chaque fichier dans votre éditeur (**Open in Visual Studio Code** ou un autre éditeur de texte)
3. Choisissez le texte à garder puis **supprimez les marqueurs** `<<<<<<<`, `=======` et `>>>>>>>`
4. Enregistrez. GitHub Desktop indique que le conflit est résolu
5. Validez la fusion par un commit puis poussez

> **Pour en avoir moins** : faites un Pull avant de travailler, des commits fréquents, des branches courtes et prévenez l'équipe quand vous modifiez un fichier commun.

Ce que GitHub Desktop fait à votre place

Dans GitHub Desktop	Commande Git	Rôle
Clone repository	`git clone <adresse>`	copier un dépôt distant
Cocher les fichiers	`git add <fichier>`	préparer le commit
Commit to main	`git commit -m "message"`	enregistrer une étape
Push origin	`git push`	envoyer sur GitHub
Pull origin	`git pull`	récupérer le travail de l'équipe
New Branch	`git switch -c <nom>`	créer une branche et s'y placer
Merge into current branch	`git merge <branche>`	fusionner une branche

À titre d'information : ces commandes se tapent dans un terminal. Vous n'en avez pas besoin dans ce module, mais vous les croiserez partout. Le livre Pro Git (en français)

#4 Travailler en équipe sur GitHub

Git et GitHub

Les issues : une tâche, un ticket

- Une **issue** est un ticket qui décrit une tâche, une idée ou un problème
- Titre, description, personne assignée (**Assignees**) et étiquettes (**Labels**)
- Onglet **Issues** → **New issue**
- `Closes #3` dans une PR ferme l'issue n° 3

#3 · Rédiger les user stories du sprint 1

documentation sprint 1

Assignée à **Manon** · Estimation : 3 points

Fermée par la PR « Ajout des user stories du sprint 1 »

> Chaque carte de votre backlog devient une issue. Le suivi du projet et le travail réel vivent au même endroit.

GitHub Projects : le tableau de bord du projet

Vue Board · le Kanban

- Les issues sont des cartes rangées en colonnes : *Todo*, *In Progress*, *Done*
- On déplace une carte quand la tâche avance
- Vision instantanée : qui fait quoi, qu'est-ce qui bloque ?

Vue Roadmap · le Gantt

- Les mêmes issues sur une frise du temps
- **Date fields** : une date de début (*Start date*) et une date de fin (*Target date*)
- **Zoom** : Month, Quarter ou Year

- Créer : onglet **Projects** → **New project**, puis relier le projet au dépôt. Une nouvelle vue s'ajoute avec **New view**
- **Milestones** (jalons) : dans **Issues** → **Milestones**, une échéance qui regroupe des issues. La Roadmap peut les afficher (**Markers**)

Documentation GitHub Projects · la vue Roadmap

Les règles d'une équipe qui travaille bien

- **Pull** avant de commencer, **push** avant de partir
- Une issue, une branche, une pull request
- Des commits fréquents aux messages clairs
- Personne ne fusionne sa propre PR sans relecture
- `main` contient toujours une version propre
- Le tableau reflète la réalité : on déplace ses cartes

> Ces règles valent aussi pour le code. Vous les appliquerez telles quelles dans vos projets de programmation.

#5 Souveraineté numérique

Git et GitHub

Où vivent vos données ?

- GitHub appartient à **Microsoft**, une entreprise américaine
- Le **Cloud Act** permet à la justice américaine d'exiger les données stockées par une entreprise américaine, même sur des serveurs situés en Europe

Souveraineté numérique : capacité à choisir et à maîtriser ses outils et ses données, sans dépendre d'un seul acteur. Le *Cloud Act* est une loi américaine de 2018.

Des alternatives ouvertes

- **GitLab** : libre dans sa version communautaire et **auto-hébergeable** sur les serveurs de l'école ou de l'entreprise
- **Forgejo** : forge libre et légère, utilisée par **Codeberg**, association à but non lucratif basée en Allemagne

> **Git reste libre.** Votre dépôt et tout son historique se déplacent d'un hébergeur à l'autre sans rien perdre. Changer d'outil est un choix, pas une impasse.

Les bons réflexes de sécurité

Votre compte

- **Double authentification** obligatoire
- Un mot de passe unique, rangé dans un gestionnaire de mots de passe
- N'invitez que les membres du projet

Votre dépôt

- **Jamais de mot de passe**, de clé d'accès ni de jeton dans un fichier versionné
- **Jamais de données personnelles** (adresses, téléphones, notes…)
- Dépôt **privé** par défaut

Supprimer un fichier ne suffit pas : **il reste dans l'historique**. Un secret poussé une fois est un secret à changer.

Ce que vous faites cet après-midi

Le dépôt

- Créé par un membre
- Équipe et Chris-CHEVALIER invités
- Cloné par chacun

Les issues

- Le backlog Trello devient des issues
- GitHub Projects : Board et Roadmap

Les livrables

- Une branche et une PR par livrable
- Relue par un autre membre

Le conflit

- Provoqué volontairement
- Résolu à deux

> Votre dépôt est un livrable noté. Il montre comment l'équipe s'est organisée.



Des questions ?

Chris CHEVALIER

chris.chevalier@ecole-hexagone.com